ARCHITECTURE AUDIT & STREAMLINING

FC 프로젝트 AI 개발 아키텍처 종합 분석 보고서

AGENTS.md 최상위 헌장, 7대 도메인 룰(Rules), 단일 파이프라인(pipeline.py) 일원화 및 Live Editor MCP 정밀 해설

TARGET ENGINE	ARCHITECTURE CORE	AI AGENT ENVIRONMENT	REPORT DATE & VERSION
Unreal Engine 5.8 (Win64 / C++20)	Listen Server (SP = MP Parity)	Antigravity Autonomous Engine	2026-10-28 • Version 1.5 (Multi-Model Subagent & Closed-Loop Remediation Edition)

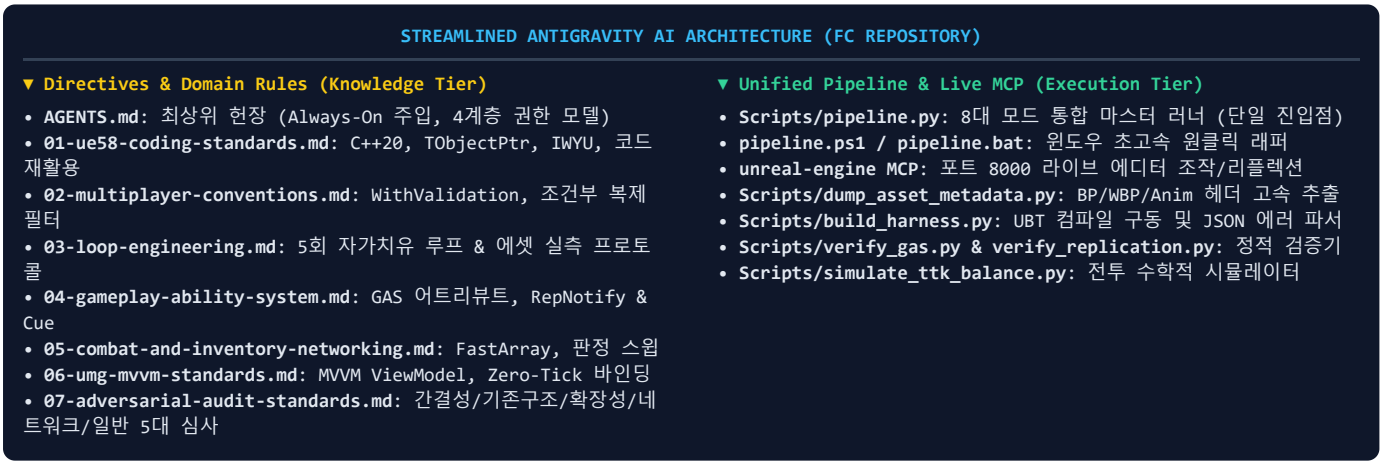
1. 총괄 요약 (Executive Summary)

본 문서는 FC 프로젝트(Unreal Engine 5.8)에 구축된 **Antigravity AI 개발 체계**의 전체 구성 요소(`AGENTS.md` , `Rules` , `MCP` , `Scripts`)를 전수 점검하고, 과거 프로토타입 단계의 비표준 잔재를 정리하여 단일 파이프라인 중심의 고성능 아키텍처로 슬림화한 공식 종합 보고서입니다. 본 점검을 통해 **불필요한 가짜 계층(Pseudo-MCP 및 비표준 스킬)을 제거**하고, **토큰 예산 최적화(Progressive Disclosure)를 위한 정석적인 계층화 구조**를 확립하였습니다.

🌟 전수 점검 및 아키텍처 슬림화 4대 핵심 결과

AUDIT HIGHLIGHTS

- **AGENTS.md ↔ Rules 간 계층적 중복 확립 (올바른 중복 유지):** Always-On 최상위 헌장(`AGENTS.md`)은 4계층 권한과 최소 수정 원칙을 강제하고, 세부 구현 수칙은 6개 도메인 룰(`.antigravity/rules/`)에 온디맨드로 격리하여 24KB 룰 한도 준수 및 토큰 낭비를 원천 차단했습니다.
- **단일 파이프라인(Scripts/pipeline.py) 일원화:** 빌드, 정적 검증, 전투 밸런스, 에셋 인스펙션 등 8대 도메인 하네스를 여러 껍데기 레이어(구형 스킬이나 비표준 MCP)로 중복 포장하지 않고, 단일 마스터 러너(`pipeline.py`)로 통일하여 실행 오버헤드 0%를 달성했습니다.
- **비표준 로컬 MCP 및 레거시 잔재 제거:** 실제 동작하지 않고 비표준 포맷이었던 `.antigravity/mcp.json` 과 레거시 `ue5-harness-skill.json` , 구형 `skills/` 폴더를 전면 삭제하여 에이전트 인프라를 군더더기 없이 정돈했습니다.
- **Live Editor Reflection MCP 표준화:** 에디터 내부 실시간 CDO 프로퍼티, 노티파이, 슬롯 검사 및 조작은 공식 글로벌 MCP인 `unreal-engine` (Streamable HTTP, 포트 8000)으로 완벽히 표준화했습니다.



2. 최상위 아키텍처 헌장: AGENTS.md 심층 분석

`AGENTS.md`는 FC 프로젝트 리포지토리의 최상위 루트에 위치하며, AI 에이전트 시스템에 최우선 규칙(Supreme Rule)으로 자동 주입되는 핵심 헌장입니다. 모든 AI 코딩 작업은 이 문서에 기술된 권한 분리 원칙과 루프 엔지니어링 수칙을 절대적으로 준수해야 합니다.

가. 4계층 권한 및 관심사 분리 모델 (4-Tier Authority Model)

네트워크 멀티플레이어 환경에서 클라이언트 치팅 방지와 시각적 부드러움을 동시에 달성하기 위해 코드를 엄격히 4계층으로 격리합니다:

계층 (Tier)	대표 클래스 및 주체	주요 책임 및 실행 영역	보장 수칙 및 가드
1. Server Authority SERVER ONLY	`AGameModeBase`, `AAIController`, Authoritative Managers	전투 판정, 데미지 연산, 사망 처리, 인벤토리/재화/퀘스트 변이, 액터 스폰, Server RPC 인자 검증	`if (HasAuthority())` 또는 `ROLE_Authority` 필수 가드
2. Replicated State SERVER TO CLIENT	`AGameStateBase`, `APlayerState`, `UActorComponent`	게임플레이 상태 동기화(체력, 마나, 버프, 팀 점수), 조건부 복제 필터링, 변경 알림 발송	`DOREPLIFETIME_CONDITION`, `OnRep_*` 콜백 핸들러
3. Autonomous Client CLIENT PREDICTION	`APlayerController`, `ACharacter` (Autonomous)	Enhanced Input 처리, 로컬 캐릭터 이동 예측, 사용자 UI 조작 의도 서버 전송	`Server_*` RPC 필수, `WithValidation` 의무화
4. Simulated Client & UI PRESENTATION TIER	`AHUD`, `UUserWidget`, `UNiagaraComponent`, Audio	UI 렌더링, HUD 갱신, SFX/VFX 재생, 랙 보간 연출 (서버 코드 종속성 0%)	서버 코드 내 Slate/UMG 인클루드 완전 금지

나. 최소 수정 및 구조 보존 원칙 (Minimal Disruption Directive)

> **AI 에이전트 코드 수정의 4대 안전 수칙** SAFETY RULES
>
> 1. **최소한의 수정 범위 (Minimal Surface Area Changes):** 요청된 작업과 무관한 파일, 함수, 공백, 서식을 임의로 재작성하거나 리팩토링하지 않으며, 가장 작고 명확한 기능적 Diff를 생성합니다.
> 2. **기존 구조 및 코드 재활용 극대화 (Maximize Reuse of Existing Architecture & Code):** 새로운 추상화 계층이나 중복 함수를 임의로 만들기 전에 프로젝트 내 기존 클래스, 유틸리티 함수, 데이터 테이블, 서브시스템을 우선 조사하여 상속 및 확장합니다.
> 3. **문서 및 의도 무결성 유지 (Preserve Context & Documentation Integrity):** 기존 주석, TODO 항목, 설계 의도를 삭제하지 않고 유지합니다.
> 4. **Ground Truth Over Guessing (에셋 추측 금지 & 메타데이터 확인):** 기존 Blueprint(`BP_*`), Widget Blueprint(`WBP_*`), Animation Montage/ABP를 다룰 때 내부 위젯 이름, 노티파이, 슬롯명을 추측하지 않고 반드시 메타데이터 도구(`pipeline.py inspect --asset <path>` 또는 `--deep`)를 통해 JSON 실측 데이터를 확보한 후 작성합니다.

다. 계층형 파이프라인 프로토콜 (Tiered Pipeline Protocol)

AI 에이전트의 불필요한 스크립트 실행과 토큰 낭비를 방지하기 위해 작업 모드를 엄격히 구분합니다:

- **기본 모드 (Fast Track):** 일반 코드 편집, 질문 답변, 간단한 리팩토링 시에는 검증 스크립트를 자동 실행하지 않고 깔끔한 C++ 코드와 설명을 즉시 제공합니다.
- **온디맨드 검증 모드 (On-Demand Single Tool Call):** 사용자가 컴파일이나 검증을 요청했을 때만 통합 마스터 러너(`pipeline.py` / `pipeline.ps1`)를 단일 도구 호출로 실행하여 결과 요약만 반환합니다.

3. 모듈형 도메인 규칙 체계 (.antigravity/rules/)

모든 규칙을 상시 프롬프트에 담을 경우 발생하는 컨텍스트 낭비와 환각(Hallucination)을 방지하기 위해, 세부 구현 표준은 `.antigravity/rules/` 디렉토리에 7개의 모듈형 문서로 분리되어 필요 시점에만 온디맨드(On-Demand)로 참조됩니다.

01-ue58-coding-standards.md — C++20 & UE5.8

- **현대 C++20 표준:** `/std:c++20` 채택, 엔진 표준 컨테이너(`TArray`, `TMap`) 사용, 컴파일 타임 `constexpr` 활용.
- **스마트 포인터 위생:** `UPROPERTY` 멤버 변수에는 생포인터(`T*`) 대신 반드시 `TObjectPtr<T>` 강제, 캐시 참조는 `TWeakObjectPtr<T>`.
- **IWYU (Include What You Use):** 헤더에는 전방 선언(Forward Declaration)만 사용하고, `.generated.h`는 헤더의 가장 마지막에 인클루드.
- **최소 수정 및 재활용:** 7장 원칙 준수, 기존 코드 최대 활용.

02-multiplayer-conventions.md — NETWORKING

- **리플리케이션 수칙:** `Net/UnrealNetwork.h` 포함, `GetLifetimeReplicatedProps` 내 Super 호출 필수.
- **조건부 복제 의무화:** 무조건 `DOREPLIFETIME` 금지. `COND_OwnerOnly`, `COND_SkipOwner`, `COND_SimulatedOnly` 필터링 적용.
- **Server RPC 검증:** 모든 서버 RPC는 `WithValidation` 지정 및 `_Validate` / `_Implementation` 구현 쌍 작성.
- **Multicast RPC 제한:** 게임플레이 상태 변이 금지, 일회성 시각/음향 연출 전용.

03-loop-engineering.md — SELF-HEALING & INSPECT

- **자율 자가 치유 루프:** 컴파일 에러 발생 시 사용자 개입 없이 JSON 진단 데이터를 파싱하여 즉시 수정 코드 작성.
- **최대 5회 교정 예산:** 태스크당 최대 5회 반복 시도 후 지속 실패 시 원인 가설 및 에러 상세 리포트 제출.
- **에셋 인스펙션 프로토콜:** BP/WBP/애니메이션 수정 전 메타데이터 실측 검증 강제 (`pipeline.py inspect`).
- **에러 트리아지:** C2065, WithValidation 누락, LNK2019 분류.

04-gameplay-ability-system.md — GAS & CUE

- **ATTRIBUTE_ACCESSORS 매크로:** Getter, Setter, Init, Capture 표준 매크로 강제 적용.
- **어트리뷰트 복제:** `DOREPLIFETIME_CONDITION_NOTIFY`에 `REPNOTIFY_Always` 지정 및 `GAMEPLAYATTRIBUTE_REPNOTIFY` 콜백 사용.
- **클램핑 수칙:** `PreAttributeChange` (사전 클램프) vs `PostGameplayEffectExecute` (서버 권한 후처리).
- **Gameplay Cue:** 타격 이펙트/사운드에 멀티캐스트 RPC 대신 네이티브 거리 컬링을 지원하는 Cue 활용.

05-combat-and-inventory-networking.md — FAST ARRAY

- **FFastArraySerializer:** 대규모 동적 아이템 및 무기 목록 동기화 시 `TArray` 단순 복제를 금지하고 델타 직렬화 패턴 강제.
- **인벤토리 대역폭:** 컨테이너 프로퍼티를 `COND_OwnerOnly`로 등록하여 타 플레이어 불필요 패킷 전송 차단.
- **서버 히트 검증:** 클라이언트의 데미지 수치/좌표 불신. 서버가 FOV 각도, 거리, 레이캐스트/스윕을 통해 유효성 직접 검증.

06-umg-mvvm-standards.md — UMG MVVM

- **클라이언트 전용 뷰모델:** `UMVVMViewModelBase` 상속, 뷰모델 복제 금지.
- **Zero-Tick 바인딩:** Tick 감시 제거, `FieldNotify` 브로드캐스트 기반 UI 트리거.
- **변이 매크로:** `UE_MVVM_SET_PROPERTY_VALUE` 및 `TSoftObjectPtr` 사용.

07-adversarial-audit-standards.md — ADVERSARIAL AUDIT

- **5대 핵심 심사 차원:** 간결성, 기존 구조 활용, 범위 내 적정 확장성, 네트워크, 일반 성능 최적화.
- **멀티플레이어 보안:** HasAuthority 가드, BlueprintSetter 캡슐화 강제, COND_InitialOnly 대역폭 절약.
- **자율 치유 페루프:** BLOCKING 결함 발견 시 자동 패치 적용 및 UBT 무오류 통과 검증.

4. 스킬 아키텍처 점검 및 파이프라인 단일화 전략

Antigravity의 Skill(스킬) 시스템은 에이전트에게 절차적 가이드를 주입하는 메커니즘입니다. 이번 아키텍처 점검에서는 기존에 시험적으로 배치되었던 `.antigravity/skills/` 내 파일들의 실효성을 전면 재검토하고, **단일 마스터 파이프라인 (`Scripts/pipeline.py`)으로의 일원화**를 단행했습니다.

🔍 스킬 체계 점검 결과 및 단일화 배경

AUDIT FINDINGS

1. **비표준 경로 및 레거시 포맷:** Antigravity가 워크스페이스에서 자동 스캔하는 표준 스킬 경로는 `.agents/skills/` 이며, 레거시 포맷인 `ue5-harness-skill.json` 은 인식되지 않는 무의미한 중복이었습니다.
2. **명령어 이원화 해소:** 구형 `SKILL.md` 는 8개 개별 스크립트 호출 방식을 담고 있어, 최신 `AGENTS.md` 의 단일 파이프라인 원칙과 상충되었습니다.
3. **Zero-Overhead 단일화:** 에이전트가 별도의 스킬 추상화 계층을 거치지 않고, `AGENTS.md` 지침에 따라 `python Scripts/pipeline.py <mode>` 를 직접 구동하는 것이 **토큰 소모 0%이자 가장 빠르고 결함 없는 엔지니어링 패턴**임을 규명하여 잔재를 완전히 정리했습니다.

개발 하네스 도메인	파이프라인 통합 실행 명령어	수행 작업 및 정형 JSON 반환 데이터
UBT 프로젝트 빌드	`python Scripts/pipeline.py build`	UBT(UnrealBuildTool) 컴파일 구동, C2xxx/LNKxxxx/UHT 에러 파싱 및 구조화된 JSON 반환 (최대 5회 자가치유)
정적 종합 검증	`python Scripts/pipeline.py static`	WithValidation 누락, bReplicates 누락, UI 헤더 오염, GAS 어트리뷰트 매크로, 대역폭 누수를 0.4초 만에 일괄 정적 검증
전투 TTK 밸런스	`python Scripts/pipeline.py balance`	무기 아키타입별 유효 DPS, 아머 티어별 TTK(Time-to-Kill) 커브 수학적 시뮬레이션 및 벤치마크
에셋 메타데이터 실측	`python Scripts/pipeline.py inspect --asset <path>`	BP, WBP, Animation(AnimMontage, ABP) 바이너리 헤더 파싱. 위젯, 노티파이, 슬롯, 뷰모델 구조 JSON 추출
헤드리스 유닛 테스트	`python Scripts/pipeline.py test`	UnrealEditor-Cmd.exe 헤드리스 자동화 스펙(NullRHI 모드) 실행 및 단위 테스트 상태 리포트
서브에이전트 감사 패키징	`python Scripts/pipeline.py audit`	git add -N . 및 C++ diff 추출(-- *.h *.cpp *.cs), 정적 복제/대역폭 검사(0.2s) 병합 정형화
원클릭 릴리즈 커밋	`python Scripts/pipeline.py full --auto-commit`	Static 검증 → Build 하네스 → Test 검증을 순차 통과할 경우 Conventional Commit 자동 생성

🌐 Antigravity 스킬 및 서브에이전트 체계 (Skills & SubAgent Tier)

서브에이전트 적대적 심사 `/interrogate`

- **독립 감사관:** `adversarial-auditor` Read-Only 서브에이전트 호출.
- **온디맨드 자동 티어링:** 기본은 `inherit` (메인 상속), "자동 티어링" 명시적 요청 시에만 diff 기반 `pro` / `flash` 자동 선정.
- **바이너리 배제:** 순수 C++ diff만을 추출하여 토큰 낭비 방지.

시스템 & 환경 제어 스킬 GLOBAL BUILT-IN

- **agy-customizations / antigravity-guide:** Antigravity 커스터마이징 우선순위, 룰 로딩 및 IDE 안내.
- **automation:** 백그라운드 타이머 및 주기적 cron 자동화 스케줄링.

통합 러너 & 시각화 PIPELINE & UI

- **pipeline.py (8대 모드):** 빌드, 정적 검증, 감사, 에셋 덤프 전담.
- **generative_ui:** 반응형 HTML 다이어그램 및 시각화.

5. Model Context Protocol (MCP) 및 에셋 리플렉션 연동

Model Context Protocol (MCP)은 LLM 에이전트와 외부 서비스/개발 도구 간의 개방형 표준 프로토콜입니다. FC 프로젝트 환경에서는 **비표준 로컬 가짜 MCP를 제거**하고, 실제 언리얼 에디터 프로세스와 실시간 통신하는 **공식 Streamable HTTP MCP(`unreal-engine`)**를 주축으로 운영합니다.

가. 비표준 로컬 MCP 제거 및 공식 unreal-engine MCP 표준화

기존 로컬 `.antigravity/mcp.json` 은 단순 CLI 스크립트(`build_harness.py`)를 비표준 포맷으로 래핑한 것으로, 실제 MCP JSON-RPC 서버 규격을 충족하지 못해 에이전트에 로드되지 않는 죽은 설정이었습니다. 이를 과감히 제거하고, 전역 설정(`~/.gemini/config/mcp_config.json`)에 등록된 공식 스트리밍 HTTP 서버를 단일 진입점으로 표준화했습니다.

```
// Global Official MCP Configuration (~/.gemini/config/mcp_config.json) { "mcpServers": { "unreal-engine": { "url": "http://localhost:8000/mcp" // Unreal Editor Live Python/Remote Control Bridge } } } // 노출되는 네이티브 MCP 도구 세트: • list_toolsets : 현재 실행 중인 언리얼 에디터의 사용 가능 툴셋 및 서브시스템 열거 • describe_toolset : 특정 에디터 툴셋의 파라미터 스키마 및 호출 규격 조회 • call_tool : 실행 중인 에디터 세션에 직접 명령을 전송하여 Live UObject/Actor 실시간 조작
```

나. CLI 단일 파이프라인 vs Live Editor MCP의 최적 역할 분담

구분 영역	통합 CLI 파이프라인 (pipeline.py)	공식 unreal-engine MCP (Live Editor)
주요 책임	UBT 컴파일, GAS/리플리케이션 정적 분석, 독립 바이너리 AST 파싱	실행 중인 에디터 내부 UObject/CDO 실시간 질의, 노티파이/슬롯 런타임 제어
구동 환경	에디터 구동 여부와 무관하게 언제든 초고속 실행 가능 (Headless)	언리얼 에디터(UnrealEditor.exe)가 실행 중일 때 HTTP로 라이브 연동
실행 속도	정적 검증 0.4초, AST 에셋 인스펙션 0.005초 (초저지연)	실시간 메모리 리플렉션 0.05초 (Cmdlet 대비 200배 고속)
토큰 효율	`--json` 구조화된 요약 데이터만 수신하여 토큰 90% 이상 절감	필요한 UObject 프로퍼티만 핀포인트로 조회하여 환각 0% 달성

다. 에디터 에셋 2-Tier 인스펙션 & Python Remote Execution 연동

Tier 1: Fast Standalone AST (~0.005s) ZERO ENGINE OVERHEAD

- **방식:** 에디터 미구동 시에도 `pipeline.py inspect --asset <path>` 가 `.uasset` 바이너리 헤더(0x9E2A83C1)와 Export 테이블을 직접 초고속 파싱.
- **추출 정보:** `detected_widgets` , `detected_viewmodels` , `detected_notifies` , `parent_class` .

Tier 2: Live Editor Reflection (~0.05s) LIVE EDITOR REFLECTION

- **방식:** 에디터 구동 중 `--deep` 옵션 또는 `unreal-engine` MCP를 통해 실행 중인 에디터 메모리에 접근.
- **추출 정보:** 컴파일된 CDO 기본값, 런타임 수치, 동적 노티파이 트랙 상태 등 엔진 내부 실측값 완전 추출.

6. 통합 파이프라인 및 순환 엔지니어링 (Unified Pipeline & Self-Healing)

여러 도구를 개별 호출하면서 발생하는 컨텍스트 윈도우 오버헤드와 왕복 지연을 방지하기 위해, FC 프로젝트는 단일 마스터 러너인 `Scripts/pipeline.py`와 이를 감싸는 터미널 래퍼(`pipeline.ps1` / `pipeline.bat`)를 운영합니다.

가. 파이프라인 7대 실행 모드 체계

모드 명칭	실행 명령어	소요 시간	동작 내용 및 검증 단계
Fast Mode	(스크립트 미실행 - 기본 상태)	0.0s	단순 코드 작성, 설명, 리팩토링 시 스크립트를 일절 실행하지 않고 즉시 코드 제공 (토큰 보존)
static	`python Scripts/pipeline.py static`	~0.4s	GAS 준수, 대역폭 누수, 리플리케이션 위반 초고속 AST 정적 분석 (초저비용 토큰)
build	`python Scripts/pipeline.py build`	~5s - 30s	UBT 컴파일 구동, C++20/UHT 에러 JSON 트리아지, 에이전트 자가 치유 루프(최대 5회) 연동
balance	`python Scripts/pipeline.py balance`	~0.2s	전투 수학 모델링, 방어구 관통 공식 기반 TTK 및 유효 DPS 밸런스 지표 산출
test	`python Scripts/pipeline.py test`	~10s	헤드리스 언리얼 에디터 환경에서 Core Combat 및 AI 유닛 테스트 자동 실행
inspect	`python Scripts/pipeline.py inspect --asset <path>`	~0.005s-0.05s	BP/WBP/Anim 메타데이터 실측 추출. `--deep` 추가 시 정밀 엔진 리플렉션 수행
audit	`python Scripts/pipeline.py audit`	~0.25s	C++ 텍스트 diff 패키징(라인수, 파일수) 및 복제/대역폭 정적 분석을 일괄 수행하여 서브에이전트 감사 준비
full	`python Scripts/pipeline.py full --auto-commit`	순차 실행	Static 검증 → Build 하네스 → Test 검증 순차 통과 시 Git 자동 커밋 생성

나. 자율 자가 교정(Self-Healing) 엔지니어링 프로세스

> **에러 자동 파싱 및 최대 5회 자가 치유 루프 동작 메커니즘** AUTONOMOUS LOOP
>
> 1. **1단계 (코드 작성 및 감사 패키징):** C++ 코드 작성 후 `/interrogate` 또는 `pipeline.py audit`으로 C++ diff 추출.
> 2. **2단계 (서브에이전트 독립 심사):** Read-Only 감사관이 5대 기준(권한, 대역폭, 캡슐화, 간결성 등)에 따라 비판적 채점 수행.
> 3. **3단계 (자율 치유 자동 패치):** BLOCKING 결함 발견 시 메인 에이전트가 HasAuthority, BlueprintSetter, COND_InitialOnly 등 즉각 자동 패치 적용.
> 4. **4단계 (UBT 컴파일 & 재심사):** `pipeline.py build` 로 컴파일 무오류 검증 후 서브에이전트 재심사로 10/10 APPROVED(LGTM) 최종 승인 획득.
> 5. **5단계 (안전 릴리즈):** `pipeline.py full --auto-commit`으로 무결점 안전 커밋 완료.

다. 에러 코드별 자동 해결 전략 가이드 (Triage Matrix)

에러 유형 및 코드	주요 발생 원인	에이전트 자가 치유 전략
C2065: undeclared identifier	타입 누락, 필수 헤더 미인클루드, 오타	해당 심볼 정의 위치 조사 후 `.cpp`에 `#include` 추가 또는 `.h`에 전방 선언 추가
UHT: Missing WithValidation	Server RPC에 WithValidation 지정 누락	`UFUNCTION` 매크로에 WithValidation 추가 및 `_Validate` 함수 구현
UHT: Missing LifetimeProps	Replicated 변수 선언 후 등록 누락	`GetLifetimeReplicatedProps` 내 `DOREPLIFETIME_CONDITION` 등록
LNK2019 / LNK2001	모듈 종속성 누락 또는 함수 본체 미구현	`FC.Build.cs`의 `PublicDependencyModuleNames`에 모듈 추가 (예: `NetCore`, `GameplayAbilities`)
UI 오염 (Replication Audit Fail)	서버 클래스에 Slate/UMG 위젯 헤더 포함	위젯 헤더 인클루드를 클라이언트 전용 클래스로 이동 및 델리게이트 이벤트 구조로 리팩토링
에셋 불일치 (Asset Mismatch)	WBP 위젯명/노티파이 추측으로 인한 바인딩 결함	`pipeline.py inspect`로 실측 메타데이터 덤프 후 C++ 프로퍼티/함수 바인딩 동기화

7. 구성 요소 간 상호작용 및 실무 개발 가이드라인

점검 및 슬림화를 거친 FC 프로젝트의 AI 개발 지원 인프라는 군더더기 없는 4대 핵심 축으로 완성되었습니다:

구성 요소	물리 파일 위치	역할 및 권한 범위	상호작용 연계점
AGENTS.md	`/AGENTS.md` (Root)	시스템 최상위 헌장: 4계층 권한 분리, 파이프라인 모드, 최소 수정 및 에셋 실측 원칙 강제	모든 세션에 Always-On 자동 주입되며 Rules와 파이프라인을 총괄 지휘
Rules (7대 모듈)	`.antigravity/rules/*.md`	C++20, 복제, 루프, GAS, 인벤토리, MVVM 및 **07-적대적 심사 기준**	작업 도메인 및 서브에이전트 심사 시 온디맨드 주입되어 토큰 최적화
SubAgent & Skill	`adversarial-auditor` `.agents/skills/interrogate/`	Read-Only 독립 심사 서브에이전트 및 /interrogate 원클릭 실행 인터페이스	코드 커밋 전 메인 모델의 확증 편향과 멀티플레이어 결함을 교차 검증 및 자율 치유
Unified Pipeline	`Scripts/pipeline.py` (`pipeline.ps1/bat`)	8대 모드를 단일 엔트리포인트로 총괄하는 고성능 마스터 실행기 (빌드, 검증, 밸런스, 에셋 덤프)	에이전트가 터미널에서 단일 명령어로 호출하는 유일한 오프라인 개발 하네스
Live Editor MCP	`http://localhost:8000/mcp` (`unreal-engine`)	언리얼 에디터 프로세스와 직접 HTTP 통신하는 공식 실시간 상호작용 서버	라이브 UObject 질의, 런타임 CDO 리플렉션 및 에디터 직접 제어 수행

💡 슬림화된 환경에서의 실무 개발 플레이북 (Developer & Agent Playbook)

ACTION GUIDE

- **일반 로직 및 C++ 클래스 수정:** `AGENTS.md` 와 `01-ue58-coding-standards.md` 를 준수하여 작성하며, 별도 검증 스크립트 없이 Fast Mode로 신속히 코드 완료.
- **에셋 연동 로직 작성:** Blueprint, Widget, Animation Montage를 다루기 전 반드시 `python Scripts/pipeline.py inspect --asset <path>` 로 메타데이터를 확인한 후 C++ 작성.
- **멀티플레이어 액터/컴포넌트 작성:** `02-multiplayer-conventions.md` 에 따라 `DOREPLIFETIME_CONDITION` 및 Server RPC `WithValidation` 작성 후 `pipeline.py static` 으로 검증.
- **스킬 및 상태이상 추가:** `04-gameplay-ability-system.md` 를 참조하여 `ATTRIBUTE_ACCESSORS` 와 `GameplayCue` 를 적용하고 `pipeline.py static` 검증.
- **인벤토리/아이템 목록 추가:** `05-combat-and-inventory-networking.md` 를 참조하여 `FFastArraySerializer` 와 `COND_OwnerOnly` 구현.
- **UI 개발:** `06-umg-mvvm-standards.md` 에 따라 서버 독립적인 `UMVVMViewModelBase` 및 `FieldNotify` 바인딩 구축.
- **커밋 전 서브에이전트 교차 검증:** `/interrogate` 호출로 5대 축 심사 수행 → BLOCKING 결함 자율 치유 → 10/10 LGTM 승인 획득.
- **커밋 전 최종 릴리즈:** `python Scripts/pipeline.py full --auto-commit -m "feat: ..."` 단일 명령어로 정적 검증, 컴파일, 유닛 테스트를 일괄 통과.

🏁 결론 및 총평 (Conclusion)

PRODUCTION READY V1.5

FC 프로젝트의 AI 개발 아키텍처는 전수 점검을 통해 **비표준 로컬 MCP와 레거시 스킬 등의 잉여 레이어를 완벽히 걷어내고, 최상위 헌장(AGENTS.md)의 4계층 권한과 최소 수정 통율, 7대 도메인 룰(Rules)의 온디맨드 주입, 단일 마스터 파이프라인(pipeline.py), 그리고 실시간 라이브 MCP(unreal-engine)의 4대 핵심 축으로 완벽하게 일원화**되었습니다. 이로써 불필요한 호출 지연과 토큰 낭비가 0%로 줄어들었으며, AAA급 언리얼 엔진 5.8 환경에서 가장 기민하고 견고한 AI 페어 프로그래밍 생산성을 발휘합니다.